

TP - CONSOMMATION D'API

B2 React.js & TypeScript - TDP 2 h - Movie Explorer

Mission. Faire évoluer Movie Explorer pour que le catalogue ne dépende plus uniquement du fichier local `movies.ts`. Les données seront récupérées depuis une API REST publique, puis affichées avec une gestion correcte du chargement et des erreurs.

API du TP. TVmaze, API publique sans clé. Nous utiliserons le catalogue des séries comme données audiovisuelles afin de travailler sur une vraie API externe.

Liste : <https://api.tvmaze.com/shows>

Détail : <https://api.tvmaze.com/shows/:id>

Important : la structure JSON de l'API n'est pas identique à notre ancien type `Movie`. Vous devrez donc comprendre les données reçues avant de les afficher.

Règles de travail

- Travail individuel à partir du projet Movie Explorer obtenu après la séance React Router.
- Documentation et IA autorisées comme outils d'aide.
- Avant de demander du code à une IA, formulez votre diagnostic ou votre hypothèse.
- Toute solution doit pouvoir être expliquée, modifiée et testée.
- Ne supprimez pas immédiatement `movies.ts` : il sert de point de comparaison avec l'ancien fonctionnement.

Résultat attendu

- Le catalogue `/movies` récupère ses données depuis l'API.
- L'interface affiche un état de chargement pendant la requête.
- Une erreur réseau/HTTP produit un message compréhensible.
- Les données reçues sont typées.
- Les cartes sont générées à partir des données distantes.
- La page `/movies/:id` récupère le détail correspondant depuis l'API.
- Un identifiant inexistant est géré proprement.
- Le projet compile avec `npm run build`.

Déroulé conseillé - 2 h

Temps	Mission	Objectif
0-15 min	Observer l'API	Lire le JSON et identifier les champs utiles.
15-30 min	Typier les données	Créer un type correspondant réellement à la réponse.
30-55 min	Charger le catalogue	<code>fetch</code> + <code>useEffect</code> + <code>useState</code> .
55-70 min	Loading	Représenter l'attente sans confondre avec une liste vide.

70-85 min	Erreurs	response.ok + try/catch/finally.
85-105 min	Détail dynamique	Charger /shows/:id depuis /movies/:id.
105-115 min	Tests	Tester succès, erreur et ID inconnu.
115-120 min	Justification	Répondre aux questions appliquées.

Mission 1 - Observer avant de coder

Ouvrez l'endpoint de liste dans le navigateur et observez le JSON. Relevez au minimum les propriétés permettant d'obtenir :

- l'identifiant
- le titre
- la date de première diffusion
- les genres
- le résumé
- l'image

À compléter avant de coder :

Besoin UI	Champ API repéré	Type probable
Identifiant		
Titre		
Année/date		
Genres		
Résumé		
Image		

Mission 2 - Créer un type pour la réponse API

Ne forcez pas l'API à ressembler artificiellement à l'ancien Movie avant de l'avoir comprise. Créez un type/interface décrivant les champs réellement utilisés.

Question : pourquoi certaines propriétés comme image peuvent-elles nécessiter null dans leur type ?

Mission 3 - Charger le catalogue

Dans Movies.tsx :

- supprimer l'utilisation directe du tableau local pour l'affichage du catalogue ;
- créer un state pour les données distantes ;
- déclencher la requête avec useEffect ;
- utiliser fetch et await ;
- stocker le JSON reçu dans le state.

Contrainte : la requête ne doit pas être lancée directement dans le corps du composant à chaque rendu.

Mission 4 - Représenter le chargement

Ajoutez un état loading. Pendant l'attente, l'utilisateur doit voir un message explicite.

Chargement du catalogue...

À expliquer : pourquoi `data.length === 0` ne permet-il pas de savoir si la requête est encore en cours ?

Mission 5 - Gérer les erreurs

- Créer un state error.
- Encadrer la requête avec try/catch/finally.
- Vérifier `response.ok` avant de lire les données comme un succès.
- Afficher un message d'erreur dans l'interface.
- Garantir que loading repasse à false en cas de succès comme en cas d'échec.

Test imposé : modifiez temporairement l'URL de l'API pour provoquer volontairement une erreur, puis remettez l'URL correcte.

Mission 6 - Adapter l'affichage

Faites évoluer MovieCard si nécessaire afin d'afficher les informations réellement disponibles : titre, année/date, genre(s), image ou résumé.

Attention : une image ou un résumé peut être absent. L'interface ne doit pas planter.

Mission 7 - Charger un détail depuis la route

La route React reste :

`/movies/:id`

Dans MovieDetails.tsx :

- récupérer id avec `useParams()`;
- déclencher une requête vers `https://api.tvmaze.com/shows/:id` ;
- faire dépendre l'effet de id ;
- gérer loading, error et la donnée de détail ;
- afficher les informations reçues.

Question clé : pourquoi [id] est-il plus cohérent que [] pour cet effet ?

Mission 8 - Tests obligatoires

Scénario	Résultat attendu	OK/KO
<code>/movies</code>	Chargement puis catalogue distant	
Rafraîchir <code>/movies</code>	Le catalogue se recharge correctement	
<code>/movies/1</code>	Détail de la ressource 1	

/movies/2	Détail différent	
Passer de 1 à 2	L'effet tient compte du changement d'id	
/movies/999999999	Erreur/ressource inexistante gérée	
URL API volontairement fausse	Message d'erreur, pas de chargement infini	
Image absente	L'interface reste stable	
npm run build	Compilation réussie	

Critères de réussite

Compétence	À renforcer	Acquis	Maîtrisé
Lire et comprendre une réponse JSON			
Typer des données externes			
Utiliser fetch avec async/await			
Déclencher un appel dans useEffect			
Raisonner sur les dépendances			
Gérer loading / data / error			
Traiter response.ok			
Charger une ressource depuis un paramètre de route			
Tester les cas d'échec			
Expliquer le code et l'usage éventuel de l'IA			

À montrer à l'enseignant

- Le projet fonctionnel.
- Le tableau d'analyse du JSON complété.
- La grille de tests complétée.
- Les réponses aux questions appliquées.
- Une explication orale d'un useEffect choisi par l'enseignant.